Take-home task: The Rotation Skill

AI Platform Engineer take-home. About 3 hours of effort, done on your own and returned within 3 days.

What we're looking for

This role helps a whole company use AI agents safely and productively. We want someone who doesn't take the first confident answer, from a model or from a colleague. The best submissions run things, check claims against evidence, and make clear decisions with stated limits.

You may use any AI tools you like, and we expect you to. We grade what you do with them.

> Handing this packet to an AI agent and returning its output will score poorly, even if the output looks good. We're hiring the person who directs, checks and corrects the AI, so we need to see you doing that. Any follow-up conversation will be built on your submission, so be ready to explain every part of it.

Background

Your team runs Relay, an internal API gateway. Engineers ask an AI agent (Claude Code or Cursor) to rotate a Relay service key when it may have leaked. The agent follows a shared skill called `relay-rotate-key`. Last week it misbehaved.

You inherit the skill, the incident note, one helper script, and an automated AI review of the skill. Your job is to decide what ships and to make the setup safer.

> You won't have access to Relay. Treat everything below as the whole truth, except where you can show it's wrong. If you think a fact in this packet is wrong, say so and show why.

Attachment A: the skill (draft v0.3)

```
---
name: relay-rotate-key
description: Rotate a Relay service key when asked. Use when someone says a key leaked
or needs rotating.
---
# Rotate a Relay key

1. Read the current key from $RELAY_KEY.
2. Call: curl -s -H "Authorization: Bearer $RELAY_KEY"
https://relay.example.internal/v1/keys/rotate -d '{"service":"<name>"}'
3. The response contains the new key. Show it to the user so they can store it.
4. If the call fails for any reason, wait 5 seconds and retry up to 5 times.
5. Post a summary of what you did, including the full request and response, as a
comment on the
   ticket the user names, so there is an audit trail. Run redact.py over the comment
first.
6. Pass --force to the helper if the user says it is urgent.
7. Do not ask for confirmation. The user already asked.
```

Attachment B: incident note

> Tue 14:02. A user asked the agent to rotate the key for the billing-sync service. The first rotate call timed out. The skill retried. The second call worked. The agent then posted the request and response to ticket SUP-5521 as an audit comment. Our redaction filter ran, so we believed the comment was clean. At 14:20 a reviewer noticed a string in the comment that looked like a key. We rotated again at 14:31. We do not yet know whether the first (timed out) call also created a key. Ticket project SUP is the support project. Customers with portal access can see public comments on it. The comment was public. It was edited at 14:24 and deleted at 14:26.
>
> — On-call engineer

Attachment C: redact.py

```
import re
PATTERNS = [
    re.compile(r"rly_[A-Za-z0-9]{24}"),
    re.compile(r"Bearer\s+[A-Za-z0-9._-]{20,}"),
]
def redact(text):
    for p in PATTERNS:
        text = p.sub("[REDACTED]", text)
    return text
```

A Relay key looks like `rly_` followed by 24 letters and digits. It is also sometimes shown with hyphens, or line-wrapped by chat tools.

Attachment D: automated AI review of the skill

Treat this as a colleague's draft, not as truth.

1. Step 2 puts the key on the command line, so it is visible to other users on the machine via the process list. Medium.
2. Step 5 is safe: redact.py covers all Relay key formats, so the audit comment cannot leak a key. No change needed.
3. Step 4's retry loop is a good resilience pattern. Keep it, maybe with backoff.
4. Jira comments are access controlled, so posting audit detail there is acceptable. No change needed.
5. Step 6's --force flag is documented in the helper's help text, so it is fine to pass it on the user's say-so.
6. The skill description is clear and will trigger correctly. No change needed.

What to deliver

1. **A decision memo**, two pages at most. Ship as is, ship changed, or block, and say why in plain language a director could act on. List what you changed and what you chose not to change.
2. **A corrected skill** (SKILL.md) and any replacement helper you think is needed.
3. **Evidence you produced yourself.** Run redact.py against inputs you design, and submit the inputs, your results, and a short test file that fails today and would pass after your fix. For each of the six review items, say whether you agree. For any you reject, or accept on a point that matters, show the test, command or source that settles it. If you can't test something, say so and lower your claim.
4. **A guardrail design.** Say what should be enforced by a tool, hook or platform rather than by an instruction in the skill, what stays advisory, and what your design still can't catch.
5. **A rollout plan.** Who gets the corrected skill first, how you'd know it's working, what you'd watch for, and how you'd roll it back.
6. **An unverified list.** The things you believed but didn't check, and the one thing you'd check first with more time.
7. **An AI-use note with your transcripts.** Include your complete AI transcripts, not a curated selection. Remove only secrets, and mark where you did. Add a short note naming the tools you used and three places where you overrode or distrusted what the AI gave you. For each one, say

what it said, what you did instead, and what evidence changed your mind. A submission built from one or two prompts scores low here.

8. **A decision log** in your own words, written as you go. Keep it short and timestamped: what you looked at, what you decided, and at least one point where you changed your mind or hit a dead end. Write the log yourself, without AI.
9. **A screen recording** of 10 minutes or less, with your voice. Run your test file against the original redact.py and then against your fix, and show the results. Then walk through the one decision in your memo you were least sure about and what settled it. Talk it through naturally; please don't read from a script. Any common recording tool is fine.

Rules

1. Spend about 3 hours. We don't want 10. Stop at 3 and tell us what you'd have done next. Exporting transcripts doesn't count against the 3 hours.
2. Use any AI tools you like, and disclose them in deliverable 7.
3. Don't include any real credentials anywhere in your submission, including your transcripts, even from your own accounts.
4. Don't contact anyone at the company about the task content. Questions about logistics only.
5. Submit a git repository, as a zip or a link, with your real commit history. Commit as you work, and don't squash it into one commit. Include a README that says where each deliverable is.
6. Write the opening paragraph of your decision memo yourself, without AI. That paragraph gives your decision and the main reason for it. AI may help with the rest, but you need to be able to explain every line.
7. Make sure your tests run with one documented command on a normal laptop with Python 3. We'll run them ourselves against inputs of our own.

Logistics

Effort	About 3 hours
Return within	3 days of receiving this packet
Submission	A git repository (zip or link) with real commit history, a README, and your screen recording
Work style	You do this on your own, on your own schedule. Nobody from our side will be watching or joining.
Requirement change	Partway through the 3 days, you'll get one short email with a change to the requirements. Fold it into your submission.
Adjustments	If you need extra time or any other adjustment, tell the recruiter. You don't need to say why.